Journal of Computational Physics 521 (2025) 113536

Contents lists available at ScienceDirect

Journal of Computational Physics

journal homepage: www.elsevier.com/locate/jcp

A generalized framework of neural networks for Hamiltonian systems

Philipp Horn [a,*], Veronica Saz Ulibarrena [b], Barry Koren [a], Simon Portegies Zwart [b]

[a] Centre for Analysis, Scientific Computing and Applications, Eindhoven University of Technology, the Netherlands
[b] Leiden Observatory, Leiden University, the Netherlands

ARTICLE INFO

MSC:
65Lxx
68T07
85-08

Keywords:
Neural networks
Hamiltonian systems
Scientific machine learning
Structure preservation
Symplectic algorithms

ABSTRACT

When solving Hamiltonian systems using numerical integrators, preserving the symplectic structure may be crucial for many problems. At the same time, solving chaotic or stiff problems requires integrators to approximate the trajectories with extreme precision. So, integrating Hamilton's equations to a level of scientific reliability such that the answer can be used for scientific interpretation, may be computationally expensive. However, a neural network can be a viable alternative to numerical integrators, offering high-fidelity solutions orders of magnitudes faster.

To understand whether it is also important to preserve the symplecticity when neural networks are used, we analyze three well-known neural network architectures that are including the symplectic structure inside the neural network's topology. Between these neural network architectures many similarities can be found. This allows us to formulate a new, generalized framework for these architectures. In the generalized framework Symplectic Recurrent Neural Networks, SympNets and HénonNets are included as special cases. Additionally, this new framework enables us to find novel neural network topologies by transitioning between the established ones.

We compare new Generalized Hamiltonian Neural Networks (GHNNs) against the already established SympNets, HénonNets and physics-unaware multilayer perceptrons. This comparison is performed with data for a pendulum, a double pendulum and a gravitational 3-body problem. In order to achieve a fair comparison, the hyperparameters of the different neural networks are chosen such that the prediction speeds of all four architectures are the same during inference. A special focus lies on the capability of the neural networks to generalize outside the training data. The GHNNs outperform all other neural network architectures for the problems considered.

1. Introduction

Specialized neural network architectures for scientific machine learning are still scarce. In most cases, neural network architectures from different areas of machine learning are used for scientific applications. If prior knowledge is included, it is usually done through additional terms to the loss function. This is also done in the Physics-Informed Neural Networks (PINNs) [1] and all of its variants.

* Corresponding author.
E-mail address: p.horn@tue.nl (P. Horn).

https://doi.org/10.1016/j.jcp.2024.113536
Received 11 August 2023; Received in revised form 21 October 2024; Accepted 24 October 2024

Available online 28 October 2024

Imposing physics constraints through the loss function does have advantages. For example, it does not require extensive expertise on neural network architectures and can be done quickly. It proves to be very useful in cases with limited data to train the neural network. However, including prior knowledge only through the loss function also has its drawbacks. Adding an additional loss term introduces a new hyperparameter, a parameter to determine the weighting between the error on the data and how far the neural network is allowed to deviate from the physics constraint. Furthermore, as hinted in the previous sentence, the neural network is not strictly following the constraint, it may deviate from it. Also, the neural network's loss only increases if the constraint is not obeyed at certain predetermined inputs called collocation points. The constraint is not enforced globally but point-wise only, and increasing the number of points slows down the training.

Including prior knowledge into the topology of the neural network requires the development of a new topology for every structure one wants to preserve. This is more labor-intensive and may ruin the approximation properties of neural network architectures for which we understand their behavior. However, if such a specialized topology can be found, the physics constraints are actually enforced everywhere; inside and outside the training data. It represents a restriction to the correct search space for the approximation instead of a change to the functional that is minimized. Furthermore, it does not require the introduction of new hyperparameters.

1.1. Hamiltonian systems

We focus on structure-preserving neural networks for Hamiltonian systems. While the neural networks analyzed in Section 5 are applicable to all Hamiltonian systems, our numerical experiments are Hamiltonian systems from mechanics. The state $\boldsymbol{x}$ of a Hamiltonian system is described by its generalized momentum $\boldsymbol{p}$ and generalized position $\boldsymbol{q}$. The system's dynamics are described by Hamilton's equations:

$$\begin{pmatrix} \dot{\boldsymbol{p}} \\ \dot{\boldsymbol{q}} \end{pmatrix} = -J \nabla H(\boldsymbol{p}, \boldsymbol{q}), \quad J = \begin{pmatrix} 0 & I_d \\ -I_d & 0 \end{pmatrix}, \quad \boldsymbol{p}, \boldsymbol{q} \in \mathbb{R}^d, \tag{1}$$

with $H : \mathbb{R}^d \times \mathbb{R}^d \to \mathbb{R}$ being the Hamiltonian of the system and I_d the d-dimensional identity matrix.

The data the neural networks are trained on consists of multiple trajectories with different initial states. The trajectory data consists of multiple states $\boldsymbol{x}_i$ along the individual trajectories that are a fixed timestep h apart from each other. The feature and label pairs of the training data are the states $\boldsymbol{x}_i$ and $\boldsymbol{x}_{i+1}$, respectively. Hence, the neural networks learn an approximation of the flow map of the system $\varphi_h : \boldsymbol{x}(t) \mapsto \boldsymbol{x}(t+h)$. This flow map can also be approximated numerically by integrators if the Hamiltonian H of the system is known.

The motivation to use neural networks for Hamiltonian systems instead of numerical integrators can be diverse. One reason could be that the Hamiltonian of the system is unknown, only data of trajectories is observed. For other Hamiltonian systems, the use of numerical integrators can be computationally expensive. This can be the case because calculating the Hamiltonian is tedious or because extremely small timesteps are required. For example in [2] the equations of motion are stiff and chaotic and therefore an accurate approximation requires small timesteps. In all these cases, the use of neural networks can be a viable alternative. Though, for the case where the Hamiltonian of the system is known and one is interested in a faster alternative to numerical integrators, the generation of the training data and the training time have to be included in the computational cost. Therefore, a neural network surrogate can only provide a reasonable speedup if the number of trajectories that have to be calculated is much larger than the training set. For the Hamiltonian systems in the numerical experiments in section 5, the Hamiltonian is known and the data is created by numerical integrators.

It is well established that symplectic integrators can be superior to non-structure-preserving numerical integrators, when applied to certain Hamiltonian systems. Especially long-term energy preservation and increased stability often are the two major benefits of symplectic integrators [3]. In symplectic integrators, the numerical flow map $\Phi_h : \boldsymbol{x}_i \mapsto \boldsymbol{x}_{i+1}$ defined by the integrator has the same geometric structure as the real flow map. This structure is given by:

$$\left(\frac{\partial \varphi_h}{\partial \boldsymbol{x}} \right)^T J \frac{\partial \varphi_h}{\partial \boldsymbol{x}} = J, \quad \boldsymbol{x} = \begin{pmatrix} \boldsymbol{p} \\ \boldsymbol{q} \end{pmatrix} \in \mathbb{R}^{2d}. \tag{2}$$

This symplectic property is the structure that is preserved in most structure-preserving neural networks for Hamiltonian systems. Some benefits of this structure have been studied thoroughly for numerical integrators. However, symplecticity in numerical integrators is not always fully understood and the benefits cannot not be carried over directly to neural networks therefore.

In this work, we give a short overview of already published neural networks for Hamiltonian systems in Section 2. In Section 3, we unite three different neural network architectures in one generalized framework called Generalized Hamiltonian Neural Networks (GHNNs). We are able to do this by presenting a new point of view on SympNets and HénonNets. This generalized framework not only includes Symplectic Recurrent Neural Networks (SRNNs), SympNets and HénonNets, but also new neural network topologies that lie beyond the capabilities of these three architectures. At the same time, the GHNNs preserve the good theoretical properties of the SympNets and HénonNets. In Section 4, we highlight important aspects related to the implementation of GHNNs. By comparing GHNNs against SympNets, HénonNets and non-structure-preserving neural networks in Section 5, we show the superior performance of a GHNN topology that cannot be categorized as SRNN, SympNet or HénonNet. For completeness, in Section 5, we also compare our GHNNs with PINN-type neural networks. All neural networks are trained with topologies that offer similar prediction speeds to achieve a fair comparison.

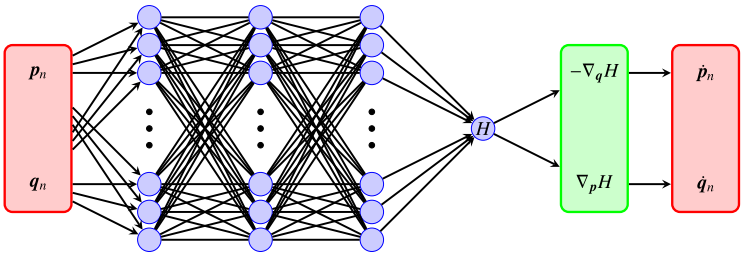

Fig. 1. Schematic of a Hamiltonian Neural Network. The input and output layers are colored red. All trainable parameters are in the blue multilayer perceptron. In the green layer the gradients of the multilayer perceptron with respect to its inputs are calculated using automatic differentiation. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

2. Previously introduced neural networks for Hamiltonian systems

Many specialized neural network architectures for data from Hamiltonian systems have already been proposed [4–8], each of them promising to surpass the previously introduced ones in numerical experiments. Moreover, each architecture tries to remove restrictions imposed on the earlier architectures. SRNNs improve on HNNs by removing the need for time derivatives in the training data and by enforcing symplecticity. SympNets remove the restriction to separable Hamiltonian systems of SRNNs and prove a universal approximation theorem. HénonNets claim to achieve a higher accuracy than SympNets by learning Hénon maps instead of unit triangular updates.

Instead of only adding to this collection of neural networks, we would like to present a generalizing framework that combines three of these types of neural networks, namely: SRNNs, SympNets and HénonNets, in a single architecture, while allowing to create new neural network topologies that lie beyond the capabilities of these other architectures. To introduce this framework, we first have a look at the already published architectures.

2.1. Hamiltonian neural networks

The first approach to neural networks specifically designed for Hamiltonian systems was presented in [4]. These Hamiltonian Neural Networks (HNN) try to learn the Hamiltonian of a system using a multilayer perceptron (MLP). To learn the Hamiltonian, not only data on positions and momenta but also data on its derivatives is needed. Then a mean-square loss can be formulated by iterating over all N datapoints:

$$L_{\text{HNN}} = \frac{1}{N}\sum_{i=1}^{N}\left\|\begin{pmatrix}\dot{\boldsymbol{p}}_i\\ \dot{\boldsymbol{q}}_i\end{pmatrix} + J\nabla H_\theta(\boldsymbol{p}_i,\boldsymbol{q}_i)\right\|_2^2. \tag{3}$$

For inference, any stable and consistent numerical integrator can be chosen to calculate $\boldsymbol{x}_{i+1}$ from $\boldsymbol{x}_i$. Also, the step size can be arbitrary since it is not part of the training. See Fig. 1, for a schematic of a Hamiltonian Neural Network.

Hamiltonian Neural Networks require additional data and the symplectic structure is only preserved if a symplectic integrator is used during inference. Furthermore, not only an approximation error in the learned Hamiltonian is accumulated, but also an additional numerical error. For these reasons, we are not going to analyze HNNs in more detail. Instead, we take a closer look at the Symplectic Recurrent Neural Networks that are heavily inspired by HNNs but without the above-mentioned shortcomings.

2.2. Symplectic recurrent neural networks

Symplectic Recurrent Neural Networks (SRNNs) were introduced in [5]. An SRNN learns a separable Hamiltonian

$$H_\theta(\boldsymbol{p},\boldsymbol{q}) = T_{\theta_1}(\boldsymbol{p}) + U_{\theta_2}(\boldsymbol{q}), \tag{4}$$

parameterized by two independent neural networks, one for the kinetic energy T_{θ_1} and one for the potential energy U_{θ_2}. The main reason behind the use of recurrent neural networks is to compensate for noisy data by using multiple combined timesteps to calculate the error. However, the important improvement over Hamiltonian Neural Networks is not reflected in the name of this architecture and is also present if SRNNs are used with simple multilayer perceptrons instead of recurrent neural networks. The important improvement is that in SRNNs the integrator is always symplectic and is embedded in the neural network's topology itself. This means that the SRNN predicts the state of the Hamiltonian system after a timestep of size h, instead of only the derivatives of the state at the current time. Therefore, a loss function can be defined with the data as introduced in Section 1.1 and no additional information of the derivatives is needed. The error is then backpropagated through the integrator into the two neural networks composing the learned Hamiltonian (see Fig. 2).

A Hamiltonian Neural Network learns a function that approximates the real Hamiltonian of the system. The approximation error together with the numerical error of the integrator, which is used for inference, add up. The SRNN directly learns the flow map of the system. As noted in [5], the learned Hamiltonian inside the SRNN adjusts to the symplectic integrator used during training and compensates for the numerical error of the integrator. Therefore, a convergence to a low value of the loss function does not imply that

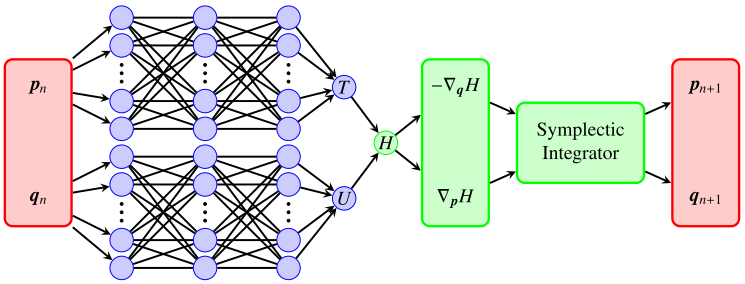

Fig. 2. Schematic of a Symplectic Recurrent Neural Network with two independent multilayer perceptrons (blue) for the learned Hamiltonian. Also, the symplectic integrator is included in the neural network itself. There are no trainable parameters in the green parts of the neural network. Instead, only predefined mathematical operations are performed.

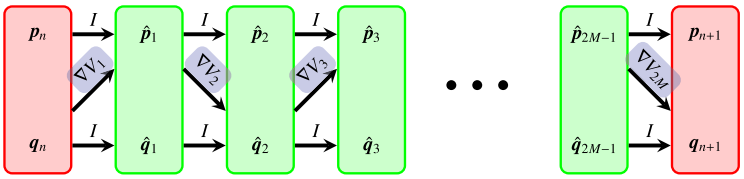

Fig. 3. Schematic of a SympNet with $2M$ modules alternating between upper and lower modules. In blue it is highlighted where the trainable parameters are located.

the learned Hamiltonian inside the neural network converges to the Hamiltonian underlying the data. Because of this behavior, the learned Hamiltonian only provides limited interpretability and it should not be used in combination with any integrator or timestep other than the one used during training.

2.3. SympNets

As a third approach we consider SympNets. SympNets have been introduced independently of the idea of learning a Hamiltonian and instead focus directly on learning symplectic maps [6]. This is done by concatenating so-called upper and lower unit triangular updates (f_{up} and f_{low}, respectively):

$$f_{\text{up}}\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix} := \begin{pmatrix}\boldsymbol{p}+\nabla V(\boldsymbol{q})\\ \boldsymbol{q}\end{pmatrix} = \begin{bmatrix} I & \nabla V(\cdot)\\ 0 & I\end{bmatrix}\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix}, \tag{5}$$

$$f_{\text{low}}\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix} := \begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}+\nabla V(\boldsymbol{p})\end{pmatrix} = \begin{bmatrix} I & 0\\ \nabla V(\cdot) & I\end{bmatrix}\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix}. \tag{6}$$

In the right hand side of equations (5) and (6) we adopted the short-hand notation of [6]. This is a slight abuse of matrix vector multiplication, which is helpful to obtain a compact notation of concatenated unit triangular updates. In every update, also called module, a different parameterized gradient ∇V is learned (Fig. 3).

Different possible parameterizations of ∇V can be introduced, the most general update is the gradient module (here as upper gradient module):

$$\nabla V(\boldsymbol{q}) := K^T \operatorname{diag}(\boldsymbol{a})\sigma(K\boldsymbol{q}+\boldsymbol{b}). \tag{7}$$

In a gradient module, the trainable parameters are: a weight matrix $K \in \mathbb{R}^{n\times d}$, a bias $\boldsymbol{b} \in \mathbb{R}^n$ and a scale factor $\boldsymbol{a} \in \mathbb{R}^n$. The width n of this module can be freely chosen. Additionally, a nonlinearity is present in the form of an activation function σ. SympNets composed of only gradient modules are called G-SympNets.

For these SympNets, a universal approximation theorem can be proven [6, Theorem 5]. In this theorem, the width and depth of the G-SympNet are not constrained. This means that, in contrast to a multilayer perceptron where one hidden layer is enough to approximate a large class of functions, a SympNet might require a large number of also wide modules to approximate a given symplectic map [9]. The proof of the universal approximation capabilities of SympNets is based on the fact that four unit triangular updates can form a Hénon-like map (defined below) and that any symplectic map can be approximated by a concatenation of these Hénon-like maps [10].

2.4. HénonNets

As an improvement to the SympNets, the HénonNets were published in [7]. Instead of learning the unit triangular updates, they directly focus on learning Hénon-like maps:

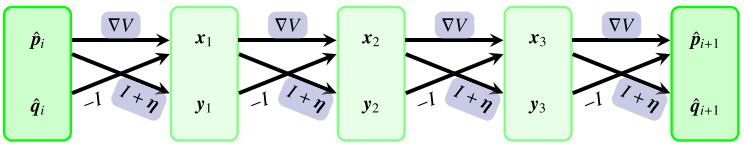

Fig. 4. Schematic of a single Hénon layer.

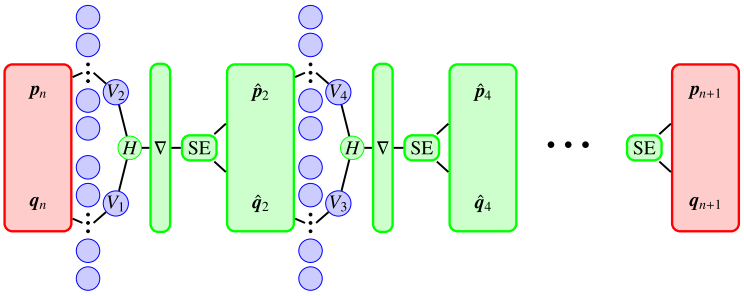

Fig. 5. A SympNet as a concatenation of Symplectic Recurrent Neural Networks.

$$h[V,\eta]\begin{pmatrix}p\\q\end{pmatrix} := \begin{pmatrix}-q+\nabla V(p)\\ p+\eta\end{pmatrix} = \begin{bmatrix}\nabla V(\cdot) & -I\\ I & 0\end{bmatrix}\begin{pmatrix}p\\q\end{pmatrix} + \begin{pmatrix}0\\\eta\end{pmatrix}. \tag{8}$$

In order to create a Hénon layer, the same Hénon-like map is iterated four times. The reasoning behind this choice is the universal approximation theorem for Hénon-like maps [10]:

$$l[V,\eta]\begin{pmatrix}p\\q\end{pmatrix} := h[V,\eta]\circ h[V,\eta]\circ h[V,\eta]\circ h[V,\eta]\begin{pmatrix}p\\q\end{pmatrix}. \tag{9}$$

In one Hénon layer, the function V and the vector $\eta \in \mathbb{R}^d$ are learned. As a parameterization for V any multilayer perceptron can be used. See Fig. 4, for a schematic of a single Hénon layer.

Since the HénonNet architecture is based on the same theory as SympNets, they are universal approximators of symplectic maps as well [7]. Because HénonNets directly focus on learning Hénon-like maps and because of the greater flexibility in the choice of the function V, HénonNets are able to achieve a lower loss than SympNets with fewer training epochs. In [7], different numerical experiments are performed to corroborate this statement.

3. Generalized framework

When analyzing the three approaches in more detail, one can see that the SRNNs, SympNets and HénonNets are actually more similar than they seem to be. This similarity allows us to introduce a generalized framework uniting all three neural network architectures. All possible SRNNs, SympNets and HénonNets are included in this generalized framework. But also, new symplectic neural network topologies can be derived from it.

A major step to show the similarities between Symplectic Recurrent Neural Networks and SympNets was made in [11]. It shows that SympNets can be seen as a concatenation of specific SRNNs. Every two gradient modules learn a separable Hamiltonian, which is parameterized by two multilayer perceptrons with one hidden layer, one multilayer perceptron for the kinetic and one for the potential energy. With this learned Hamiltonian, one step of a Symplectic Euler (SE) method is performed. The next two layers learn a different Hamiltonian, perform another Symplectic Euler step and so on. This new point of view on SympNets is illustrated in Fig. 5.

A similar equivalence exists between HénonNets and SympNets and therefore also between HénonNets and SRNNs. We can show that one Hénon layer is equivalent to a concatenation of four specific unit triangular updates:

$$l[V,\eta]\begin{pmatrix}p\\q\end{pmatrix} = \begin{bmatrix}I & \nabla V_4(\cdot)\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ \nabla V_3(\cdot) & I\end{bmatrix}\begin{bmatrix}I & \nabla V_2(\cdot)\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ \nabla V_1(\cdot) & I\end{bmatrix}\begin{pmatrix}p\\q\end{pmatrix}, \tag{10}$$

with:

$$V_1 = f\circ V\,, \text{ where } f(x) = -x, \tag{11a}$$

$$V_2 = V\circ f\,, \tag{11b}$$

$$V_3 = f\circ V\circ g\,, \text{ where } g(x) = -x-\eta, \tag{11c}$$

$$V_4 = V\circ h\,, \text{ where } h(x) = x-\eta. \tag{11d}$$

Table 1
Overview of three of the different neural networks for Hamiltonian systems that were introduced in the literature so far.

	SRNNs	SympNets ($2n$ gradient modules)	HénonNets (n Hénon layers)
Definition of the input to output mapping	$\mathrm{SRNN}(\boldsymbol{p},\boldsymbol{q}) = \mathrm{SI}(H_{\mathrm{NN}},\boldsymbol{p},\boldsymbol{q})$	$\mathrm{SympNet}(\boldsymbol{p},\boldsymbol{q}) = \mathrm{SE}(H_{\mathrm{NN}_n},\cdot,\cdot)\circ\cdots\circ \mathrm{SE}(H_{\mathrm{NN}_1},\boldsymbol{p},\boldsymbol{q})$	$\mathrm{HénonNet}(\boldsymbol{p},\boldsymbol{q}) = \mathrm{SE}(H_{\mathrm{NN}_{2n}},\cdot,\cdot)\circ\cdots\circ \mathrm{SE}(H_{\mathrm{NN}_1},\boldsymbol{p},\boldsymbol{q})$
Details of the learned Hamiltonians	$H_{\mathrm{NN}}(\boldsymbol{p},\boldsymbol{q}) = T_{\mathrm{NN}}(\boldsymbol{p}) + U_{\mathrm{NN}}(\boldsymbol{q})$	$H_{\mathrm{NN}_i}(\boldsymbol{p},\boldsymbol{q}) = T_{\mathrm{NN}_i}(\boldsymbol{p}) + U_{\mathrm{NN}_i}(\boldsymbol{q})$	$H_{\mathrm{NN}_{2i-1}}(\boldsymbol{p},\boldsymbol{q}) = V_{\mathrm{NN}_i}(-\boldsymbol{p}) + V_{\mathrm{NN}_i}(\boldsymbol{q})$ and $H_{\mathrm{NN}_{2i}}(\boldsymbol{p},\boldsymbol{q}) = V_{\mathrm{NN}_i}(-\boldsymbol{p}-\boldsymbol{\eta}) + V_{\mathrm{NN}_i}(\boldsymbol{q}-\boldsymbol{\eta})$
Parameterization of the learned Hamiltonians	T_{NN} and U_{NN} can be any multilayer perceptrons.	T_{NN_i} and U_{NN_i} are multilayer perceptrons with one hidden layer.	V_{NN_i} can be any multilayer perceptron.
The integrators that can be used	SI can be any symplectic integrator that is explicit for separable Hamiltonians.	SE is the Symplectic Euler integrator.	SE is the Symplectic Euler integrator.

The derivation of this expression can be found in Appendix A.

In a HénonNet, V is parameterized by any multilayer perceptron. Hence, V_1, V_2, V_3 and V_4 are given by the same neural network, only with slight modifications in their weights and biases:

- Multiplying the weights and biases in the output layer of V by -1 yields V_1.
- Multiplying the weights in the first hidden layer of V by -1 yields V_2.
- First subtracting $W_1\boldsymbol{\eta}$ from the biases in the first hidden layer of V, then multiplying the weights in the first hidden layer by -1 and finally multiplying the weights and biases in the output layer by -1 yields V_3.
- Subtracting $W_1\boldsymbol{\eta}$ from the biases in the first hidden layer of V yields V_4.

Here, W_1 is the weight matrix of the first hidden layer in V.

Therefore, HénonNets consist of the same unit triangular updates as SympNets. The main difference is that per Hénon layer one arbitrary multilayer perceptron is trained and four unit triangular updates are performed with slight modifications of this multilayer perceptron. Note that since modifications of the same multilayer perceptron are reused, the four unit triangular updates are not independent and share weights and biases. Since SympNets are, due to the unit triangular updates, equivalent to a concatenation of SRNNs and because HénonNets are built out of the same unit triangular updates, HénonNets are equivalent to a concatenation of SRNNs as well.

An overview of the three different neural network architectures is given in Table 1. All three approaches are phrased as possibly multiple integration steps with learned Hamiltonians. Although SRNNs were originally introduced this way, we just saw that SympNets and HénonNets can be formulated in a similar fashion. Moreover, this highlights the fact that not only SRNNs learn a Hamiltonian, but SympNets and HénonNets learn multiple hidden Hamiltonians as well. Whether those Hamiltonians approximate the underlying Hamiltonian of the system is not clear. The purpose of presenting all approaches in this formulation is to showcase the similarities and to discuss the differences.

The point of view in which layers correspond to integration steps for learned ordinary differential equations is not new. It is similar to the one introduced for ResNets in [12]. Moreover, it is used to analyze the stability of neural networks in [13] and to later introduce Neural Ordinary Differential Equations [14]. Also, it is interesting to note that approximating symplectic maps by concatenating integration steps for Hamiltonian systems is close to the key idea in the proof that any symplectic map can be approximated by a concatenation of Hénon-like maps [10]. This is what both the universal approximation theorems of the SympNets and HénonNets are based on.

Alternatively, one could write all neural network architectures as concatenations of unit triangular updates. This is more in line with how SympNets and HénonNets are introduced but it is also possible for SRNNs. Since symplectic integrators that are explicit for separable Hamiltonian systems alternate between updating positions and momenta, they also perform unit triangular updates [15]. The formulation as unit triangular updates shows the similarities as well. However, it is less elegant for the SRNNs because which updates are performed and how the learned maps V are connected depends on the symplectic integrator that is used. In both formulations it becomes clear how one can create a generalized framework covering all these neural networks for Hamiltonian systems and even enabling to introduce new ones.

3.1. Generalized Hamiltonian neural networks

To get an overview of how the different methods intersect, we present Fig. 6, where it is shown that there is already overlap between the different methods. Any SympNet with only two gradient modules is also an SRNN and any SRNN using the symplectic Euler method and only one hidden layer in both multilayer perceptrons is also a SympNet. Furthermore, any HénonNet using only one hidden layer in all multilayer perceptrons for the learned maps V is also a SympNet (but with additional constraints).

We propose a novel type of neural networks for Hamiltonian systems. This new neural network architecture we call Generalized Hamiltonian Neural Networks (GHNNs). All types of neural networks for Hamiltonian systems discussed so far can be found as special

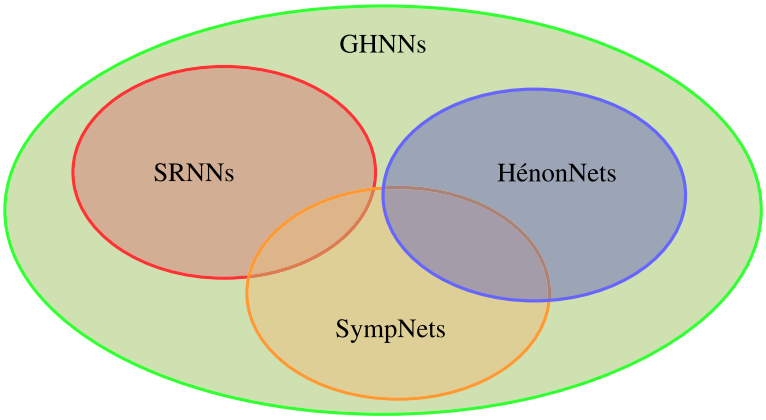

Fig. 6. Venn diagram showing the overlap between the different neural network architectures for Hamiltonian systems.

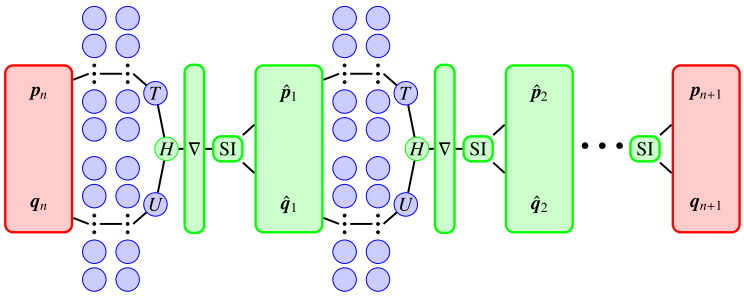

Fig. 7. Schematic of a Generalized Hamiltonian Neural Network.

cases of this framework. This is achieved by removing as many restrictions as possible from the individual layers, while still keeping the symplectic structure in the topology of the neural network.

In the same formulation as used in Table 1, in which layers are considered to be integration steps with learned Hamiltonians, the new architecture can be written as:

$$\mathrm{GHNN}(\boldsymbol{p},\boldsymbol{q}) = \mathrm{SI}_n(H_{\mathrm{NN}_n},\cdot,\cdot)\circ\cdots\circ\mathrm{SI}_1(H_{\mathrm{NN}_1},\boldsymbol{p},\boldsymbol{q}), \quad H_{\mathrm{NN}_i}(\boldsymbol{p},\boldsymbol{q}) = T_{\mathrm{NN}_i}(\boldsymbol{p}) + U_{\mathrm{NN}_i}(\boldsymbol{q}), \tag{12}$$

with T_{NN_i} and U_{NN_i} being any multilayer perceptrons and SI_i any symplectic integrator that is explicit for separable Hamiltonian systems.

To recognize the SRNNs as special cases of the GHNNs one has to restrict the GHNNs to one symplectic integrator ($n = 1$). The SympNets can be found in the GHNNs by using the symplectic Euler method for all symplectic integrators SI_i and only using one hidden layer in all multilayer perceptrons of all the learned Hamiltonians H_{NN_i}. Building a HénonNet from this general framework requires more restrictions. First, again only the symplectic Euler method can be used. Second, an even number of integrators has to be used. Finally, while general multilayer perceptrons can be used for the Hamiltonians, many weights and biases are shared between the kinetic and potential energies and also between two different Hamiltonians. The details of this weight sharing are provided in Table 1 and the list after Equations (11a-d).

Consequently, one way to create neural networks that are not included in one of the three existing architectures would be to use an integrator different from the symplectic Euler method. Or, instead of using a different integrator, another approach could be to use more than the one hidden layer in the multilayer perceptrons that compose the learned Hamiltonians in SympNets. One such GHNN with two hidden layers is shown in Fig. 7.

In SympNets, only one hidden layer is used since this suffices to prove the universal approximation theorem. However, from machine learning literature we know that using deep neural networks instead of wide ones can lead to better approximation capabilities with fewer trainable parameters [16].

4. Implementation

Contrary to how we introduced GHNNs, we do not implement them using the concept of learned Hamiltonians and performing integration steps. While SRNNs were implemented that way, it requires calculating gradients of the neural networks with respect to their input during training and also during inference. On the one hand, this can be done with automatic differentiation, but on the other hand, it considerably slows down the prediction speed of the neural network. For that reason, we instead precalculate the

Table 2
Topology hyperparameters of the different neural networks used for the numerical experiments. The number of layers (except for MLP) is the number of layers of the MLPs that compose the learned Hamiltonians.

NN type	Learned Hamiltonians	Layers	Neurons per Layer	Trainable parameters		
				pendulum	double pendulum	3-body problem
MLP	-	5	128	66690	67204	69260
SympNet	10	1	50	3000	4000	8000
HénonNet	10	1	50	755	1010	2030
GHNN	5	2	25	7250	7500	8500
Deep HénonNet	6	2	25	2178	2256	2568

gradients of multilayer perceptrons with one and two hidden layers. The same is done in the original implementation of the gradient modules in SympNets. This does not imply a restriction of the GHNNs. Mathematically, they perform the same calculations as if they were implemented as concatenated integrators. It simply enables a better comparison with SympNets. For a fair comparison we also use this implementation for HénonNets.

Calculating the gradient of a multilayer perceptron with one hidden layer is straightforward. The result can be found in [6] for example:

$$\nabla_{\boldsymbol{q}}\mathrm{MLP}(\boldsymbol{q}) = \nabla_{\boldsymbol{q}}\boldsymbol{w}^T\Sigma(W\boldsymbol{q}+\boldsymbol{b}) = W^T\mathrm{diag}(\boldsymbol{w})\sigma(W\boldsymbol{q}+\boldsymbol{b}). \tag{13}$$

Here $W \in \mathbb{R}^{n\times d}$ refers to the weight matrix in the hidden layer, $\boldsymbol{b}\in\mathbb{R}^n$ to the bias vector in the hidden layer, $\boldsymbol{w}\in\mathbb{R}^n$ to the weight matrix/vector in the output layer and Σ to the activation function, with σ its derivative. Both are applied element-wise. In this calculation, $\boldsymbol{q}$ can be exchanged with $\boldsymbol{p}$ for the gradient needed for a lower triangular update.

The gradient of a multilayer perceptron with two hidden layers is far more convoluted:

$$\nabla_{\boldsymbol{q}}\mathrm{MLP}(\boldsymbol{q}) = \nabla_{\boldsymbol{q}}\boldsymbol{w}^T\Sigma(W_2\Sigma(W_1\boldsymbol{q}+\boldsymbol{b}_1)+\boldsymbol{b}_2) = W_1^T\mathrm{diag}\left(W_2^T\mathrm{diag}(\boldsymbol{w})\sigma(W_2\Sigma(W_1\boldsymbol{q}+\boldsymbol{b}_1)+\boldsymbol{b}_2)\right)\sigma(W_1\boldsymbol{q}+\boldsymbol{b}_1). \tag{14}$$

Here, $W_1\in\mathbb{R}^{n\times d}$ refers to the weight matrix and $\boldsymbol{b}_1\in\mathbb{R}^n$ to the bias vector in the first hidden layer. $W_2\in\mathbb{R}^{m\times n}$ refers to the weight matrix in the second hidden layer and $\boldsymbol{b}_2\in\mathbb{R}^m$ to its bias vector. Again, $\boldsymbol{w}\in\mathbb{R}^m$ is the weight matrix/vector in the output layer and the activation functions are the same as before. The derivation of this gradient is detailed in Appendix B.

The gradients of multilayer perceptrons with three or more hidden layers can be calculated as well. However, because the expression then becomes even more complicated, we compute these gradients using automatic differentiation. For our numerical experiments in the next section, only multilayer perceptrons with up to two layers are used for a fair computation speed comparison with the other neural network architectures.

All code to train the neural networks for this project is written in Python using PyTorch [17]. The code including methods to generate and process the data for the numerical experiments can be found on GitHub (https://github.com/AELITTEN/GHNN). Our data can also be found on Zenodo (https://zenodo.org/records/11032352) and the 850 neural networks trained for this work are available in a separate GitHub repository (https://github.com/AELITTEN/NeuralNets_GHNN).

5. Numerical experiments

In order to compare the new Generalized Hamiltonian Neural Networks with existing neural networks for Hamiltonian systems and with physics-unaware neural networks as well, we perform multiple experiments. For a fair and objective comparison it is not obvious how to choose the topology hyperparameters of the different neural networks (such as the number of neurons and the number of layers). Instead of comparing neural networks with a similar number of trainable parameters, we compare neural networks with similar prediction speeds during inference and training. Due to the computationally expensive need to calculate the gradients of the learned Hamiltonians inside SRNNs, comparing SRNNs to the other neural networks is unfair. As a consequence, we do not include SRNNs in the experiments.

The hyperparameters of the neural networks that lead to similar prediction speeds, and are therefore used for the numerical experiments in this section, can be found in Table 2. For all different types of neural networks, including GHNNs, it is not obvious how to choose the hyperparameters. As a starting point for the SympNet hyperparameters, we take a look at the SympNets from [6]. To guarantee that we do not restrict the capabilities of the SympNets, we use the largest SympNets of this paper, which are the ones used for the 3-body problem considered in [6]. Since GPUs are used for the training of the neural networks and for inference and since large matrix-vector products in each layer can be efficiently parallelized, the main hyperparameter determining the speed is the depth of the neural network and not the number of neurons in each layer. Hence, the total number of independent layers in the SympNets and GHNNs is kept the same. The SympNets have one hidden layer per multilayer perceptron, two multilayer perceptrons per learned Hamiltonian and ten learned Hamiltonians. The GHNNs have two hidden layers per multilayer perceptron, two multilayer perceptrons per learned Hamiltonian and five learned Hamiltonians. A HénonNet performs four sequential operations with one trained multilayer perceptron. It has one independent multilayer perceptron per two learned Hamiltonians. Therefore, only one quarter of the total number of independent layers of a SympNet can be used in a HénonNet to end up at the same prediction speed. The deep HénonNets are HénonNets with two hidden layers per multilayer perceptron. These were not tested in the original paper that introduced the HénonNets [7]. Since a physics-unaware multilayer perceptron does not need a large depth to be a universal approximator, only five layers are used.

While the general framework of our GHNNs allows the use of any symplectic integrator that is explicit for separable Hamiltonians, only the Symplectic Euler method is used for the numerical experiments. Usually a higher-order accurate integrator like the Störmer-Verlet integrator leads to higher accuracy in the prediction of the next state. But, since the Hamiltonians are only learned and do not necessarily correlate with the real Hamiltonian of the system, a better performance of a GHNN with a higher-order accurate integrator cannot be expected. Using the Störmer-Verlet integrator would definitely decrease the prediction speed and might also lead to problems during the training since the unit triangular updates are no longer independent, which complicates the optimization. Nevertheless, investigating the use of other integrators could be an interesting topic for future research.

How the total number of trainable parameters depends on depth, width and the dimension of the Hamiltonian system widely varies for the different neural network architectures. This can be seen in the last three columns in Table 2. The number of trainable parameters heavily depends on the dimension of the Hamiltonian system for SympNets and HénonNets. On the other hand, for the MLP and GHNN the dimension of the Hamiltonian system only has a small effect. This can be explained by the fact that the number of trainable parameters in both an MLP and a GHNN scales with n^2:

$$\text{Trainable parameters in an MLP} = (l-1)n^2 + (4d+l)n + 2d, \tag{15a}$$

$$\text{Trainable parameters in a SympNet} = 2m(d+2)n, \tag{15b}$$

$$\text{Trainable parameters in a HénonNet} = m/2((l-1)n^2 + (d+l+1)n + d) \overset{l=1}{=} m/2((d+2)n + d), \tag{15c}$$

$$\text{Trainable parameters in a GHNN} = 2m((l-1)n^2 + (d+l+1)n), \tag{15d}$$

where l refers to the number of hidden layers, n to the number of neurons per layer, d to the dimension of the Hamiltonian system (state $\boldsymbol{x}$ is in $\mathbb{R}^{2d}$) and m is the number of learned Hamiltonians, i.e., half the number of gradient modules or twice the number of Hénon maps, respectively.

For each experiment, 50 neural networks per architecture are trained using the Adam algorithm [18]. Loss plots for all neural networks and all numerical experiments are provided in Appendix C. All 50 of these neural networks sharing the same architecture are identical except for the seeds in the random initialization of their weights. These weights are drawn from a normal distribution with a mean of 0 and a variance of 0.1. All biases are initialized as 0. The PyTorch seed is set to 1 for the first neural network, to 2 for the second, and so on. Resulting in different but repeatable initial weights for all neural networks with the same architecture. This allows a quantification of the influence of the initialization of the neural networks and enables us to be certain about whether one architecture is better than the other (given our chosen hyperparameters).

For all three experiments, error plots are provided to compare the different architectures. For these figures, the mean absolute error over the trajectories $\left\{ \begin{pmatrix} \boldsymbol{p}_i^T(t) & \boldsymbol{q}_i^T(t) \end{pmatrix}^T \mid i \in S_{\text{test}}, t \le t_{\text{end}} \right\}$ in the test data S_{test} is calculated for all neural networks. For a fixed time t the mean $\mu(t)$ over all 50 neural networks is calculated as:

$$\mu(t) = \frac{1}{50} \sum_{j=1}^{50} \mu_j(t), \tag{16}$$

with

$$\mu_j(t) = \frac{1}{2d|S_{\text{test}}|} \sum_{i \in S_{\text{test}}} \left\| \begin{pmatrix} \boldsymbol{p}_{i,\text{data}}(t) \\ \boldsymbol{q}_{i,\text{data}}(t) \end{pmatrix} - \begin{pmatrix} \boldsymbol{p}_{i,\text{NN}_j}(t) \\ \boldsymbol{q}_{i,\text{NN}_j}(t) \end{pmatrix} \right\|_1 . \tag{17}$$

Additionally, as a measure of the variance due to different random initializations of the weights and biases, an area with the upper and lower bounds given by the 90% and 10% quantiles of $\mu_j(t)$ at each point in time is plotted in the figures.

5.1. Weakly-symplectic neural networks

In addition to the various structure-preserving neural networks, we train a multilayer perceptron with a modified loss function.

$$L = \underbrace{\frac{1}{2d\,N_{\text{train}}} \sum_{i=1}^{N_{\text{train}}} \left|\left| \boldsymbol{x}_{i,\text{lab}} - \text{MLP}(\boldsymbol{x}_{i,\text{feat}}) \right|\right|_2^2}_{\text{MSE}} + \lambda \underbrace{\frac{1}{(2d)^2 N_{\text{symp}}} \sum_{i=1}^{N_{\text{symp}}} \left|\left| \left(\frac{\partial \text{MLP}}{\partial \boldsymbol{x}}\bigg|_{\boldsymbol{x}_i} \right)^T \boldsymbol{J} \, \frac{\partial \text{MLP}}{\partial \boldsymbol{x}}\bigg|_{\boldsymbol{x}_i} - \boldsymbol{J} \right|\right|_{\text{F}}^2}_{L_{\text{symp}}} . \tag{18}$$

In this modified loss function, a term L_{symp}, is multiplied by a weighting factor λ and added to the mean square error (MSE) on the trajectory data, to enforce the symplectic structure in a weak sense. In the calculation of the mean square error, $\left\{ \left(\boldsymbol{x}_{i,\text{feat}}, \boldsymbol{x}_{i,\text{lab}} \right) \mid i = 1, \ldots, N \right\}$ are the pairs of features and labels in the training data. In the calculation of the symplectic loss term, N_{symp} is the number of collocation points in which we enforce the symplectic structure. Further, $\frac{\partial \text{MLP}}{\partial \boldsymbol{x}}\Big|_{\boldsymbol{x}_i}$ is the Jacobian of the multilayer perceptron with respect to its inputs at the points $\boldsymbol{x}_i = \begin{pmatrix} \boldsymbol{p}_i^T & \boldsymbol{q}_i^T \end{pmatrix}^T$ in phase space. Finally, $||\cdot||_{\text{F}}$ in (18) is the Frobenius norm. The points in which the symplectic structure is enforced are arranged on a regular grid and different numbers of these points and values for the weighting factor λ are tested.

This method of adding prior knowledge through an additional regularizing loss term is similar to a PINN [1]. The main difference is that adding only the symplectic structure instead of the ODE/PDE residual does not require knowledge of the equations describing the dynamics of the system. Also, adding only a symplectic constraint enables a fair comparison against the structure-preserving neural networks in which also only the symplectic structure is preserved.

Since the amount of points in a grid in $2d$ dimensions grows exponentially with the number of dimensions, we are only training these weakly-symplectic neural networks for the single pendulum. This one experiment suffices to see the benefits and drawbacks of including prior physical knowledge of the data in the loss function.

5.2. Pendulum

The most common and simple Hamiltonian system used for testing different neural network architectures is the mathematical pendulum. It consists of a rigid, massless rod hanging from a pivot with a point mass in a gravitational field at its end. The pendulum is allowed to move freely in two-dimensional space, without friction and drag. The generalized position of this system is the angle of the pendulum. The dynamics, given an initial angle q and initial momentum p, are described by Hamilton's equations (1) with the following Hamiltonian:

$$H(p,q) = \frac{p^2}{2ml^2} + mgl(1-\cos(q)), \tag{19}$$

with m the mass at the end of the rod, l the length of the rod and g the gravitational acceleration. One can simplify the system by assuming $g = 1$ and by also setting l and m equal to 1. Furthermore, shifting H by a constant, which does not change the dynamics of the system since only the gradients of the Hamiltonian are of importance, allows H to become:

$$H(p,q) = \frac{p^2}{2} - \cos(q). \tag{20}$$

To create the dataset, 500 random initial angles $q_0 \in [-0.95\pi, 0.95\pi]$ are picked uniformly and the initial momenta are set to 0. This corresponds to a pendulum that is raised to the initial angle q_0 and then released. For all 500 initial conditions, the different trajectories are calculated with a step size $h = 0.01$ using the Symplectic Euler method. The states for the training data are recorded every 10 steps, so effectively with a step size $h = 0.1$. Since the trajectories of a pendulum are periodic, the final time is chosen individually for each trajectory to be a period $t_{\text{end}} = T$. For the training data, the trajectories are cut after $t_{\text{end,train}} = 0.25T$ to assess the generalization capabilities of the different neural network architectures. The neural networks are therefore only trained on data of falling pendulums. Once the angle 0 is reached, the rest of the data is removed from the training data. In the p-q phase space, this translates to no training data being in the quadrants with positive p and positive q as well as negative p and negative q. Overall, the data is split into 80% training data, 10% validation data and 10% test data.

As mentioned in Section 5.1, we also train weakly-symplectic neural networks on the pendulum data. We train two different versions of these multilayer perceptrons, always training again 50 differently initialized neural networks. The first version (lower left plots in Fig. 8) adds a symplectic loss at only 16 points and with a weighting factor of 1. These points are on a grid with 4 equidistant points in the p-interval $[-1.5, 1.5]$ and 4 equidistant points in the q-interval $[-3, 3]$. Additionally, we train 50 MLPs with a symplectic loss with a weighting factor of 100 and 81 points at which the symplectic loss is calculated. These points are again on an equidistant grid but with 9 points in the p-interval $[-2.5, 2.5]$ and 9 points in the q-interval $[-3, 3]$. Hence, the second version of weakly-symplectic MLPs (lower right plots in Fig. 8) enforces the symplectic structure more strongly and in a slightly larger area of the phase space. The value of λ is typically optimized using a cross-validation technique instead of being chosen arbitrarily by the person training the neural networks. However, these two values ($\lambda = 1$ and $\lambda = 100$) are enough to show the trade-offs that one has to keep in mind when training these types of neural networks.

As can be seen in Fig. 8, all six neural networks are able to learn the dynamics quite well inside the training data range. However, Fig. 9 shows that there is a large difference between how well the predicted trajectories match the ground truth, even inside the training data. Especially, the effect of the low number of trainable parameters in the HénonNets becomes apparent. Outside the training data the error of the multilayer perceptron diverges exponentially in a few steps. Our GHNN on the other hand has a much smaller error for the entire trajectory of one period. It is interesting to see that although the errors of the SympNet and HénonNet after $t = 0.25T$ are large as well, the error between $t = 0.5T$ and $t = 0.75T$ (the second area with training data) results from a phase shift in the predictions. This is visible in the energy plots in Fig. 8. This phase shift is accumulated outside the training data range. Once the pendulum reaches its highest point on the left (minimal angle), it re-enters the training data. The trajectories predicted by the SympNet and HénonNet enter the training data range at the correct point in phase space, but take more iterations to do so. Hence, a phase shift is accumulated.

Both Fig. 8 and Fig. 9 show that the added symplectic loss in the weakly-symplectic multilayer perceptrons helps the neural networks to generalize outside the training data. The generalization capabilities of these neural networks however are outmatched by the SympNets and particularly by our GHNNs. For the weakly-symplectic multilayer perceptron with $\lambda = 100$, a reduced accuracy can be observed inside the training data. The neural network has to use some of its approximation capabilities to reduce the symplectic loss and trades this against a higher loss on the data.

To understand the effect of the added symplectic loss, in Fig. 10, we visualize the symplectic error in phase space for all three types of multilayer perceptron. Even the multilayer perceptron without an added symplectic loss is learning the symplectic structure from the data in all parts of the phase space where there is training data. If the symplectic loss is only added at 16 points and with

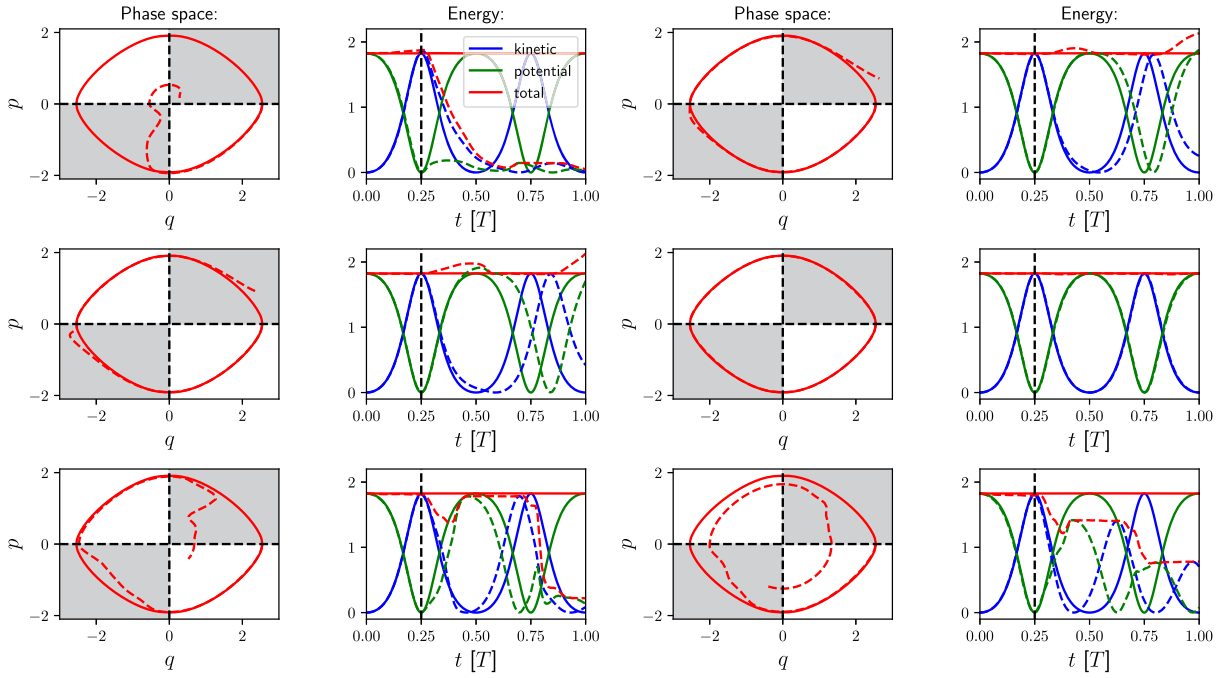

Fig. 8. Comparison of the predictions of a MLP, SympNet, HénonNet, GHNN and two weakly-symplectic MLPs (from top left, to right, to bottom right) for one period of the single pendulum with an initial angle of about 0.8π. The solid lines are the data and the dashed lines the predictions. The black dashed line indicates where the training data ends. In the gray areas there is no training data.

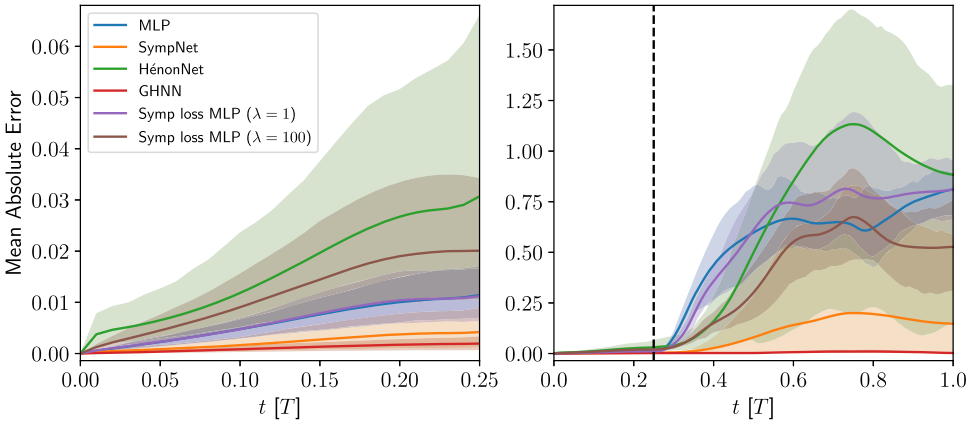

Fig. 9. Comparison of the mean error over all the trajectories in the test data of the single pendulum for 50 MLPs, SympNets, HénonNets, GHNNs and different weakly-symplectic MLPs. The lighter colored areas indicate the variations between the differently initialized neural networks. The left graph is inside the time range included in the training data. The right graph shows one full period with the black dashed line indicating where the time range included in the training data ends.

a weight $\lambda = 1$ the neural network also only learns the structure at these 16 points locally. With 81 points and a weight $\lambda = 100$ the multilayer perceptron starts to also have a low symplectic error in between the points.

We conclude that enforcing the symplectic structure of the single pendulum in the neural network strongly improves the generalization properties of the neural network. Furthermore, for the single pendulum, the GHNN with two hidden layers for every learned Hamiltonian outperforms all other approaches inside and outside the training data by several orders of magnitude. Trying to learn the symplectic structure by adding an additional loss term is already challenging for this simple Hamiltonian system. To see a clear benefit in the generalization capabilities one has to enforce the structure on a fine grid. For a higher dimensional Hamiltonian system or a system in which the states are located in a larger area in the phase space, this approach is not feasible. Instead of using a fine grid, one can also use methods like (quasi) Monte Carlo to sample the collocation points. This leads to considerably better scaling to higher dimensions. However, it still requires a good knowledge of the relevant area of the phase space to make sure it is covered completely.

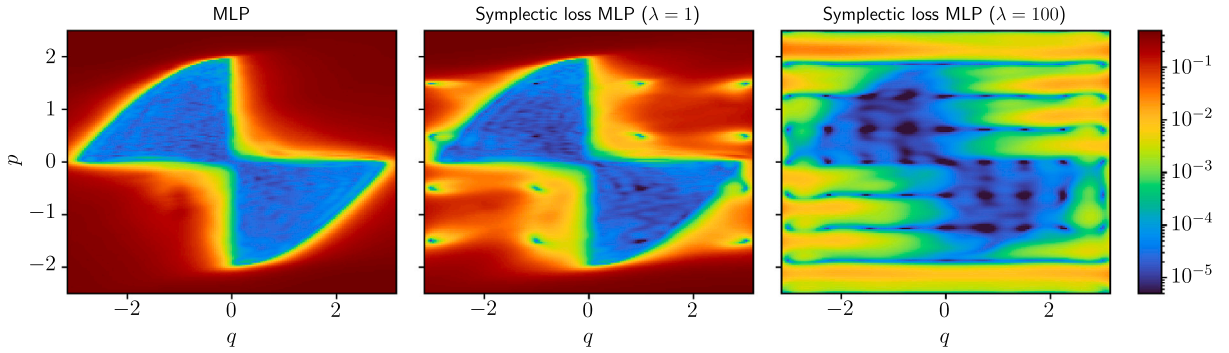

Fig. 10. Mean symplectic error in the phase space of the single pendulum of multilayer perceptrons with three different loss functions. The mean of the symplectic error is taken over 50 differently initialized neural networks per type. In the left plot, no symplectic constraint is added to the loss. Going to the right a stronger symplectic constraint is added to the loss.

5.3. Double pendulum

A considerably more complicated Hamiltonian system is the double pendulum. It is built similarly to the single pendulum but with a second pivot and a second rod hanging from the first one, and also another point mass being attached to the second rod. The double pendulum is considered to move freely in two-dimensional space without friction and drag. The generalized positions for this system are the two angles of the double pendulum.

For this two-dimensional system, the Hamiltonian is more lengthy and convoluted. More importantly, the Hamiltonian of this system is no longer separable:

$$H(\boldsymbol{p},\boldsymbol{q}) = \frac{m_2 l_2^2 p_1^2 + (m_1+m_2) l_1^2 p_2^2 - 2m_2 l_1 l_2 p_1 p_2 \cos(q_1-q_2)}{2m_2 l_1^2 l_2^2 (m_1 + m_2 \sin^2(q_1-q_2))} + (m_1+m_2) g l_1 (1-\cos(q_1)) + m_2 g l_2 (1-\cos(q_2)) + m_1 g l_2. \tag{21}$$

We simplify the Hamiltonian again by choosing all parameters (g, m_1, m_2, l_1, l_2) to be equal to 1. With a shift by a constant value the Hamiltonian then simplifies to:

$$H(\boldsymbol{p},\boldsymbol{q}) = \frac{p_1^2 + 2p_2^2 - 2p_1 p_2 \cos(q_1-q_2)}{2 + 2\sin^2(q_1-q_2)} - 2\cos(q_1) - \cos(q_2). \tag{22}$$

The double pendulum is not only of higher dimension and has a non-separable Hamiltonian, it also is a chaotic system. We use 2000 trajectories instead of 500 (single pendulum) to train the neural networks. The two random initial angles of these trajectories are drawn uniformly from $[-0.5\pi, 0.5\pi]$. The initial momenta are again zero and the trajectories are recorded with a step size $h = 0.1$. To calculate the trajectories, again the Symplectic Euler method is used to ensure that the data actually belongs to a symplectic map, even though higher-order accurate non-symplectic integrators might result in trajectories closer to the ground truth. So, here we take an approach opposite to the one followed in [7] where the standard fourth-order Runge-Kutta method (RK4, [19]) is used instead of a symplectic integrator. We adapt the step size during the integration according to a convergence criterion, using intermediate steps that are not recorded for training. For this criterion, we divide the step size in half and double the amount of steps repeatedly until the calculated state $\boldsymbol{x}_{i+1} \approx \boldsymbol{x}(t+h)$ stops changing with smaller step sizes. For the training data the trajectories are cut after $t_{\text{end,train}} = 5$, while the full trajectories are calculated until $t_{\text{end}} = 10$ to analyze generalization. The split into training, validation and test data is the same as for the pendulum (80% training data, 10% validation data and 10% test data).

Fig. 11 demonstrates the superiority of the symplectic neural networks. The energy in the predictions of the SympNet and GHNN oscillates similarly to the one of the symplectic integrator. The energy predicted by the HénonNets also oscillates around the true energy but with a larger amplitude. The energy in the MLP's predictions is stable inside the training time range but then suddenly plummets after $t = 5$, i.e., outside the training data for this particular trajectory.

Because there is no periodicity in the trajectories of the double pendulum and since we train on trajectory data, the distinction between inside and outside the range of training data is not as clear as for the single pendulum. However, because all trajectories start with zero initial momenta, most of the training features are concentrated around large angles $\boldsymbol{q}$ and small momenta $\boldsymbol{p}$. The features for $t > 5$ are spread over wider ranges of $\boldsymbol{q}$ and $\boldsymbol{p}$ than for $t < 5$. Fig. 12 supports this statement.

In contrast to Fig. 11, there is not such a dramatic increase in the error of the MLP after $t = 5$ in Fig. 13. Meaning, on average over all trajectories in the test data, the MLP does not perform as badly as for the one trajectory in Fig. 11. As already mentioned, this can be explained by the lack of a sharp transition between training data and non-training data. Fig. 13 shows the drawback of the low number of trainable parameters in the HénonNets. The error is about six times larger than the error of the other symplectic neural networks and higher than the one of the MLP inside the training data. The errors of the SympNets and GHNNs are comparable. While the mean over the errors of all 50 neural networks is lower for the SympNets, the lighter colored areas largely overlap, indicating that this is caused by the random initialization of the different neural networks. The higher errors of all methods inside the training data can be explained by the chaotic nature of the double pendulum and consequently the considerably more complicated flow map of this system.

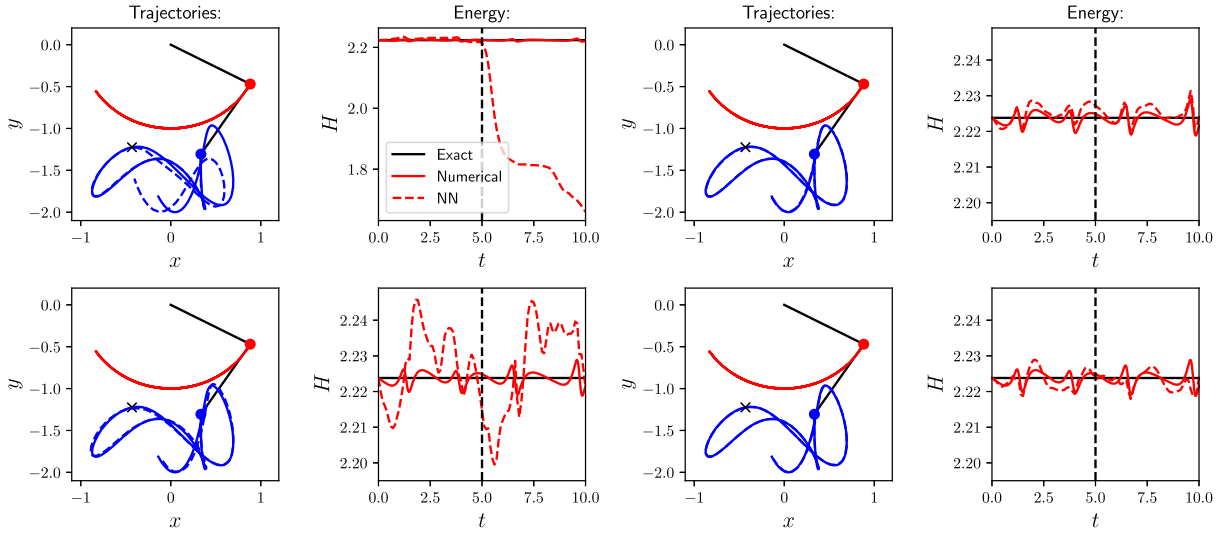

Fig. 11. Comparison of the predictions of one MLP, SympNet, HénonNet and GHNN (from top left, top right, to bottom right) for one exemplary test trajectory of the double pendulum. The solid lines are the data and the dashed lines the predictions. The black dashed line in the energy plots and the black cross in the trajectory plots indicate where the training data ends.

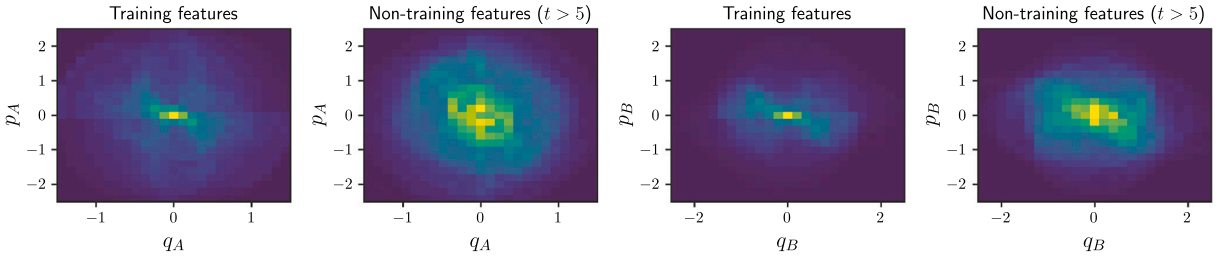

Fig. 12. This histogram shows how the features in the double pendulum training data are distributed, compared to how they are distributed in the fraction of the data that is not part of the training data. Yellow indicates a high number of points in the corresponding bin and dark blue a low number. The indices A and B distinguish the two rods with point masses at their ends. A is the inner one and B the outer one that is attached to A.

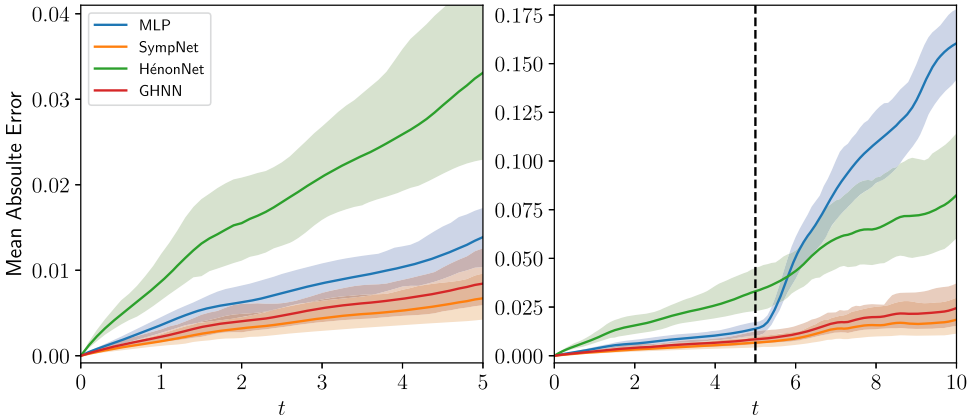

Fig. 13. Comparison of the mean error over all the trajectories in the test data of the double pendulum for 50 MLPs, SympNets, HénonNets and GHNNs. The lighter colored areas indicate the variations between the differently initialized neural networks. The left graph is inside the time range included in the training data. The right graph shows the error along the full trajectories with the black dashed line indicating where the time range included in the training data ends.

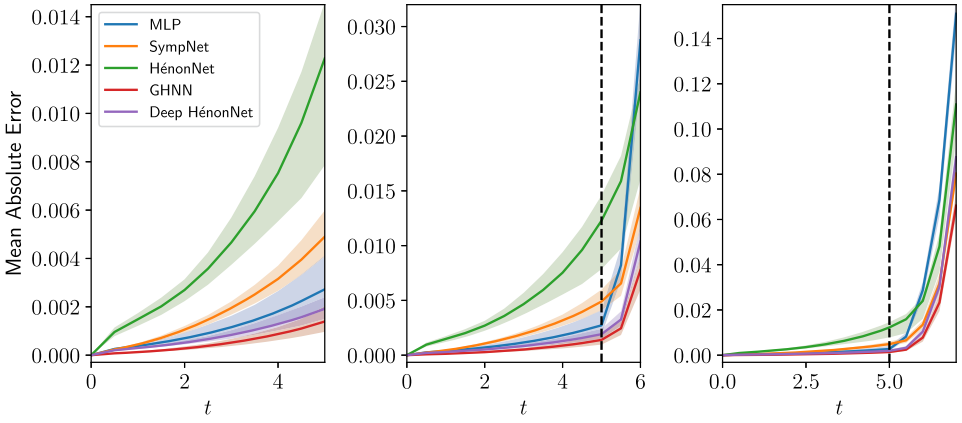

Fig. 14. Comparison of the mean error over all the trajectories of the three bodies, in the test data for 50 MLPs, SympNets, HénonNets and GHNNs. Additionally to the HénonNets with one hidden layer per learned V, the error of 50 HénonNets with two hidden layers is shown (Deep HénonNet). The lighter colored areas indicate the variations between the differently initialized neural networks. The left graph shows the time range included in the training data. The middle graph shows the error inside the time range included in the training data and two steps outside, with the black dashed line indicating where the time range included in the training data ends. The right graph shows the errors along the full trajectories.

5.4. 3-Body problem

As a third test case we compare the neural networks on data of a gravitational 3-body problem. In this Hamiltonian system, three bodies of different masses orbit each other according to Newton's law of gravitation. The gravitational N-body problem is important for astrophysics. The dynamics of planets, stars and galaxies are described by this Hamiltonian system as long as no relativistic effects are considered.

The system, in all three spatial dimensions, is of the total dimension $d = 3N$, with $\boldsymbol{q} = \left(\boldsymbol{q}_1^T \cdots \boldsymbol{q}_N^T\right)^T \in \mathbb{R}^d$ and the canonical coordinates of each body $\boldsymbol{q}_i$ being the coordinates in Euclidean space. The Hamiltonian of the gravitational N-body problem can be written as

$$H(\boldsymbol{p}, \boldsymbol{q}) = \frac{1}{2}\boldsymbol{p}^T M^{-1}\boldsymbol{p} - \sum_{i=1}^{N}\sum_{j=1}^{i} G\frac{m_i m_j}{||\boldsymbol{q}_j - \boldsymbol{q}_i||_2}, \tag{23}$$

where G is the gravitational constant and M the mass matrix, which consists of the masses m_i of the N bodies and is given in three-dimensional space by $M := \mathrm{diag}(m_1, m_1, m_1, m_2, \cdots, m_N, m_N, m_N)$.

For our numerical experiments, we simplify the problem. First of all, we restrict ourselves to the 3-body problem in two spatial dimensions; the overall system is six-dimensional. Moreover, we choose all masses to be dimensionless and all equal to one; $m_i = 1$. Further, we scale the system to $G = 1$ [20]. This results in

$$H(\boldsymbol{p}, \boldsymbol{q}) = \frac{1}{2}\boldsymbol{p}^T\boldsymbol{p} - \sum_{i=1}^{3}\sum_{j=1}^{i}\frac{1}{||\boldsymbol{q}_j - \boldsymbol{q}_i||_2}, \quad \text{with: } \boldsymbol{q}_i \in \mathbb{R}^2. \tag{24}$$

The data of the 3-body system is generated using the arbitrarily high-precision code Brutus [21]. This enables us to make sure that the data the neural networks are trained on is actually the ground truth. Since the 3-body problem is a chaotic problem, the initial conditions are chosen such that the trajectories start with rather simple and easy to predict dynamics that become increasingly more chaotic over time. The initial conditions are the same as in the 3-body data of the papers introducing HNNs and SympNets [4,6].

The random initial positions of the bodies are on a circle around the origin with the radius r drawn uniformly from $[0.9, 1.2]$. Furthermore, all three bodies have the same initial distance from each other. With these initial positions, one can calculate the momenta of the three bodies such that they would stay on the circle with radius r, orbiting the origin. These momenta are multiplied by different random uniform factors from $[0.8, 1.2]$ and chosen as initial momenta. With the initial conditions arranged in such a way, only small accelerations and slow exchange in kinetic and potential energy occur until $t \approx 5$. Afterwards, close encounters of the bodies may occur, leading to high accelerations and quick exchange in potential and kinetic energy. As in [6], 5000 trajectories are generated, a final time $t_{\text{end,train}} = 5$ and a step size $h = 0.5$ are used for training. In contrast to [6], a final time $t_{\text{end}} = 7$ is used for testing to check whether the neural networks can generalize from the simple behavior to the case with close encounters. The data is split into 80% training, 10% validation and 10% test data.

The benefit of the high number of trainable parameters inside a multilayer perceptron becomes apparent in Fig. 14. Inside the training data the multilayer perceptrons have an error about five times lower than the SympNets. However, the middle graph then directly reveals the most severe drawback of the physics-unaware multilayer perceptrons. After only two timesteps outside of training data, the error of the predictions by the multilayer perceptrons is higher than the error of the four physics-aware neural networks.

Moreover, the error of the GHNNs inside the training data is five times lower than the error of the multilayer perceptrons. Hence, the GHNNs are highly accurate and able to generalize at the same time. Again, the HénonNets with one hidden layer per MLP have the highest error of all neural networks inside the training data and are able to generalize slightly better than the multilayer perceptrons outside the training data.

The deep HénonNets with two hidden layers per MLP perform considerably worse than all other neural networks in the previous experiments and therefore were not included for the pendulum and double pendulum. The high error of the deep HénonNets is the result of only three Hénon layers being used to achieve the same prediction speed as all the other neural networks. Since a large depth might be required by the structure-preserving neural networks in order to approximate certain symplectic maps, only three Hénon layers might be insufficient. For the 3-body problem, however, the roles are reversed. The deep HénonNets have a smaller error than all other methods except for the GHNN, both inside and outside the training data. This shows that to approximate the flow map of the 3-body problem, more complex updates of the state are of greater importance than many updates. For the same reason, the GHNNs outperform the SympNets by an order of magnitude.

6. Conclusion

We introduced an overarching framework that unites three widely used neural network architectures specifically designed to learn the symplectic flow maps of Hamiltonian systems: SRNNs, SympNets and HénonNets. Additionally, this framework allows for the creation of novel neural network topologies that cannot be built from the individual architectures. These new Generalized Hamiltonian Neural Networks (GHNNs) combine the benefits and flexibility of all three aforementioned neural network architectures.

In the numerical experiments, 50 neural networks of four different architectures (MLP, SympNet, HénonNet and GHNN) were trained with the hyperparameters chosen such that a prediction is realizable at the same speed for all networks. In these experiments, our neural networks, the GHNNs, outperform all other neural networks for the single pendulum data. The extrapolation capabilities of the GHNNs are astonishingly good. The errors of the GHNNs and the SympNets are similar when trained on data for the double pendulum system. Of both, the errors are remarkably smaller than those of HénonNets and multilayer perceptrons. For the 3-body problem, the GHNNs have better accuracy than all other neural networks, with the errors being over one order of magnitude smaller than those of the SympNets. This drastic variation in the difference between the errors of the GHNNs and SympNets (also HénonNets and deep HénonNets) suggests that there are symplectic maps, where in order to approximate these, complex unit triangular updates are essential and others where many simple ones are the better choice. Due to the second layer in the MLPs of the GHNNs and deep HénonNets, far more complex unit triangular updates can be learned by these methods.

These numerical experiments show that a universal approximation theorem and a high number of trainable parameters are insufficient to guarantee good approximation capabilities. A universal approximation theorem can be proven for all three symplectic neural network architectures and the number of trainable parameters in the SympNets and GHNNs for the 3-body problem are almost the same. We conclude that it is important how the trainable parameters are arranged in the neural network. The choice of how to arrange the trainable parameters can decrease the prediction error more than one order of magnitude while preserving the same calculation speed.

At the same time, the HénonNets demonstrate that having too few trainable parameters also limits the accuracy of the predictions. In a comparison with a similar number of trainable parameters for all neural network architectures, as done in [7], HénonNets might outperform other approaches. In addition, all neural networks in our numerical experiments were trained for many epochs to ensure that a good optimum was found in the loss landscape. This allowed for a fair comparison of the approximation capabilities of the different architectures. In [7], the number of epochs is quite limited and the HénonNets have, opposite to our findings, a smaller error than the SympNets. This indicates that the additional structure in the HénonNets might be helpful in reaching a good approximation quickly but restricts the approximation when the neural network is trained for a long time.

Overall, the added symplecticity in all structure-preserving neural networks yields a better generalization and stability outside the training data. Moreover, the structure-preserving architectures achieve comparable or better accuracy than the multilayer perceptrons with far fewer trainable parameters. This can be useful for the application to large Hamiltonian systems where large neural networks are needed and memory is limited.

CRediT authorship contribution statement

Philipp Horn: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Methodology, Investigation, Formal analysis, Data curation, Conceptualization. **Veronica Saz Ulibarrena:** Writing – review & editing, Validation. **Barry Koren:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **Simon Portegies Zwart:** Writing – review & editing, Validation, Supervision, Software.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was funded by the Dutch Research Council (NWO), in the framework of the program 'Unraveling Neural Networks with Structure-Preserving Computing' (file number OCENW.GROOT.2019.044).

Appendix A. Relation between a Hénon layer and unit triangular updates

The following is a detailed calculation proving that one Hénon layer is equivalent to the concatenation of four specific unit triangular updates. Again the same notation as introduced in [6] is used for a compact notation of unit triangular updates. The first line is the definition of a Hénon layer. In the following four lines, all four Hénon maps are calculated. Afterwards, four unit triangular updates are identified, line by line:

$$
\begin{aligned}
l[V,\boldsymbol{\eta}]\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix} &= h[V,\boldsymbol{\eta}]\left(h[V,\boldsymbol{\eta}]\left(h[V,\boldsymbol{\eta}]\left(h[V,\boldsymbol{\eta}]\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix}\right)\right)\right)\\
&= h[V,\boldsymbol{\eta}]\left(h[V,\boldsymbol{\eta}]\left(h[V,\boldsymbol{\eta}]\begin{pmatrix}-\boldsymbol{q}+\nabla V(\boldsymbol{p})\\ \boldsymbol{p}+\boldsymbol{\eta}\end{pmatrix}\right)\right)\\
&= h[V,\boldsymbol{\eta}]\left(h[V,\boldsymbol{\eta}]\begin{pmatrix}-\boldsymbol{p}-\boldsymbol{\eta}+\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p}))\\ -\boldsymbol{q}+\nabla V(\boldsymbol{p})+\boldsymbol{\eta}\end{pmatrix}\right)\\
&= h[V,\boldsymbol{\eta}]\begin{pmatrix}\boldsymbol{q}-\nabla V(\boldsymbol{p})-\boldsymbol{\eta}+\nabla V(-\boldsymbol{p}-\boldsymbol{\eta}+\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p})))\\ -\boldsymbol{p}+\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p}))\end{pmatrix}\\
&= \begin{pmatrix}\boldsymbol{p}-\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p}))+\nabla V(\boldsymbol{q}-\nabla V(\boldsymbol{p})-\boldsymbol{\eta}+\nabla V(-\boldsymbol{p}-\boldsymbol{\eta}+\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p}))))\\ \boldsymbol{q}-\nabla V(\boldsymbol{p})+\nabla V(-\boldsymbol{p}-\boldsymbol{\eta}+\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p})))\end{pmatrix}\\
&= \begin{bmatrix}I & \nabla V(\cdot-\boldsymbol{\eta})\\ 0 & I\end{bmatrix}\begin{pmatrix}\boldsymbol{p}-\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p}))\\ \boldsymbol{q}-\nabla V(\boldsymbol{p})+\nabla V(-\boldsymbol{p}+\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p}))-\boldsymbol{\eta})\end{pmatrix}\\
&= \begin{bmatrix}I & \nabla V(\cdot-\boldsymbol{\eta})\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ \nabla V(-\cdot-\boldsymbol{\eta}) & I\end{bmatrix}\begin{pmatrix}\boldsymbol{p}-\nabla V(-\boldsymbol{q}+\nabla V(\boldsymbol{p}))\\ \boldsymbol{q}-\nabla V(\boldsymbol{p})\end{pmatrix}\\
&= \begin{bmatrix}I & \nabla V(\cdot-\boldsymbol{\eta})\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ \nabla V(-\cdot-\boldsymbol{\eta}) & I\end{bmatrix}\begin{bmatrix}I & -\nabla V(-\cdot)\\ 0 & I\end{bmatrix}\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}-\nabla V(\boldsymbol{p})\end{pmatrix}\\
&= \begin{bmatrix}I & \nabla V(\cdot-\boldsymbol{\eta})\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ \nabla V(-\cdot-\boldsymbol{\eta}) & I\end{bmatrix}\begin{bmatrix}I & -\nabla V(-\cdot)\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ -\nabla V(\cdot) & I\end{bmatrix}\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix}\\
&= \begin{bmatrix}I & \nabla V_4(\cdot)\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ \nabla V_3(\cdot) & I\end{bmatrix}\begin{bmatrix}I & \nabla V_2(\cdot)\\ 0 & I\end{bmatrix}\begin{bmatrix}I & 0\\ \nabla V_1(\cdot) & I\end{bmatrix}\begin{pmatrix}\boldsymbol{p}\\ \boldsymbol{q}\end{pmatrix},
\end{aligned}
$$

with:

$$
\begin{aligned}
V_1 &= f\circ V\,,\ \text{where } f(\boldsymbol{x}) = -\boldsymbol{x},\\
V_2 &= V\circ f\,,\\
V_3 &= f\circ V\circ g\,,\ \text{where } g(\boldsymbol{x}) = -\boldsymbol{x}-\boldsymbol{\eta},\\
V_4 &= V\circ h\,,\ \text{where } h(\boldsymbol{x}) = \boldsymbol{x}-\boldsymbol{\eta}.
\end{aligned}
$$

Appendix B. Calculation of the gradient of a two-layer MLP

The output of a multilayer perceptron with two hidden layers can be calculated by:

$$
\begin{aligned}
\mathrm{NN}(\boldsymbol{q}) &= \boldsymbol{a}^T\Sigma(\tilde{K}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{\boldsymbol{b}})\\
&= \sum_{i=1}^{m} a_i\Sigma(\tilde{K}_{i\cdot}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{b}_i)\\
&= \sum_{i=1}^{m} a_i\Sigma\left(\sum_{j=1}^{n}\tilde{K}_{ij}\Sigma(K_{j\cdot}\boldsymbol{q}+b_j)+\tilde{b}_i\right)\\
&= \sum_{i=1}^{m} a_i\Sigma\left(\sum_{j=1}^{n}\tilde{K}_{ij}\Sigma\left(\sum_{l=1}^{d}K_{jl}q_l+b_j\right)+\tilde{b}_i\right).
\end{aligned}
\tag{B.1}
$$

Here, $K \in \mathbb{R}^{n\times d}$ and $\tilde{K} \in \mathbb{R}^{m\times n}$ are the weight matrices in the first and second hidden layer, respectively, and $\boldsymbol{a} \in \mathbb{R}^m$ is the weight vector of the output layer. Further, $\boldsymbol{b} \in \mathbb{R}^n$ and $\tilde{\boldsymbol{b}} \in \mathbb{R}^m$ are the biases in the first and second hidden layer. The activation function Σ is applied element-wise in the hidden layers.

The partial derivative in the direction of the s-th dimension of $\boldsymbol{q}$ can be computed to be:

$$\begin{aligned}\frac{\partial}{\partial q_s}\mathrm{NN}(\boldsymbol{q}) &= \sum_{i=1}^{m} a_i\sigma(\tilde{K}_{i.}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{b}_i)\cdot\left(\sum_{j=1}^{n}\tilde{K}_{ij}\sigma(K_{j.}\boldsymbol{q}+b_j)K_{js}\right)\\ &= \sum_{i=1}^{m} a_i\sigma(\tilde{K}_{i.}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{b}_i)\cdot((K^T)_{s.}\mathrm{diag}(\tilde{K}_{i.})\sigma(K\boldsymbol{q}+\boldsymbol{b})).\end{aligned} \tag{B.2}$$

Overall, this implies that the full gradient is given by:

$$\begin{aligned}\nabla\mathrm{NN}(\boldsymbol{q}) &= \sum_{i=1}^{m} a_i\sigma(\tilde{K}_{i.}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{b}_i)K^T\mathrm{diag}(\tilde{K}_{i.})\sigma(K\boldsymbol{q}+\boldsymbol{b})\\ &= K^T\left(\sum_{i=1}^{m} a_i\sigma(\tilde{K}_{i.}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{b}_i)\mathrm{diag}(\tilde{K}_{i.})\right)\sigma(K\boldsymbol{q}+\boldsymbol{b})\\ &= K^T\mathrm{diag}\left(\sum_{i=1}^{m} a_i\sigma(\tilde{K}_{i.}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{b}_i)\tilde{K}_{i.}\right)\sigma(K\boldsymbol{q}+\boldsymbol{b})\\ &= K^T\mathrm{diag}\left(\tilde{K}^T\mathrm{diag}(\boldsymbol{a})\sigma(\tilde{K}\Sigma(K\boldsymbol{q}+\boldsymbol{b})+\tilde{\boldsymbol{b}})\right)\sigma(K\boldsymbol{q}+\boldsymbol{b}).\end{aligned} \tag{B.3}$$

Appendix C. Loss during training of the different architectures

The training behavior of the different neural network architectures is given in Fig. C.15, which shows that the SympNets and the GHNNs achieve a similar loss for the single and double pendulum. However, the GHNNs need less than 500 epochs to reach a good minimum in the loss landscape, whereas the SympNets take around 1500 epochs to find a similar minimum.

In Fig. C.16, the loss is plotted over all 25000 epochs. It is clear that after 5000 epochs, the loss of almost all neural networks for all three Hamiltonian systems has nearly converged. After 5000 epochs, only the loss of the GHNNs on the pendulum data and the loss of the MLPs on the 3-body problem data keep improving significantly.

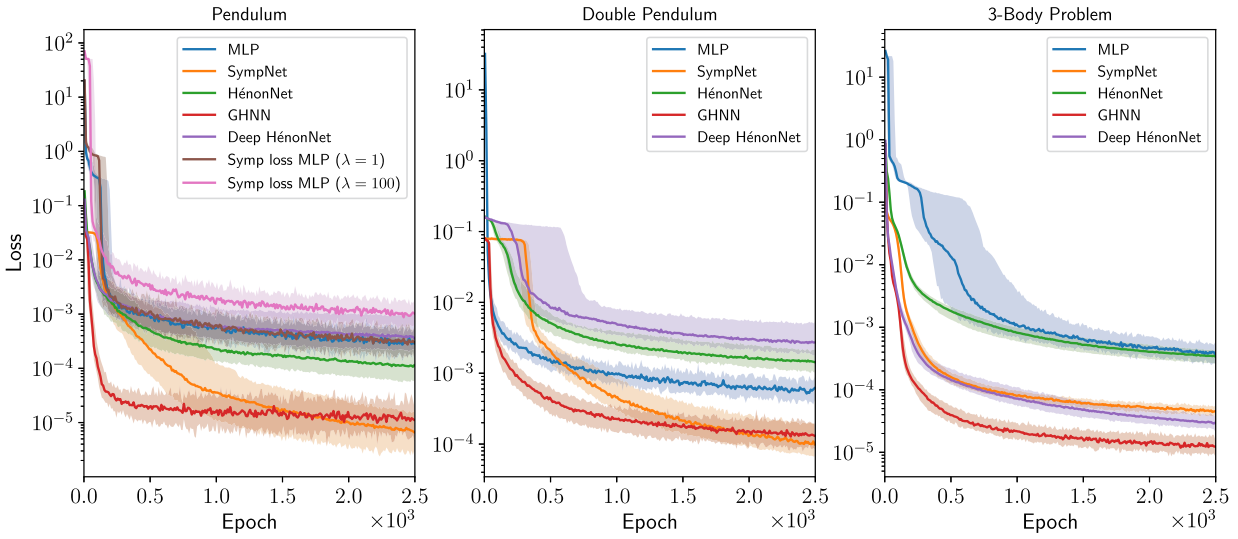

Fig. C.15. Comparison of the validation loss during the first 2500 epochs of training for all three Hamiltonian systems. The solid lines show the behavior of the median of the 50 neural networks trained per architecture. The lighter colored areas indicate the variations between the differently initialized neural networks.

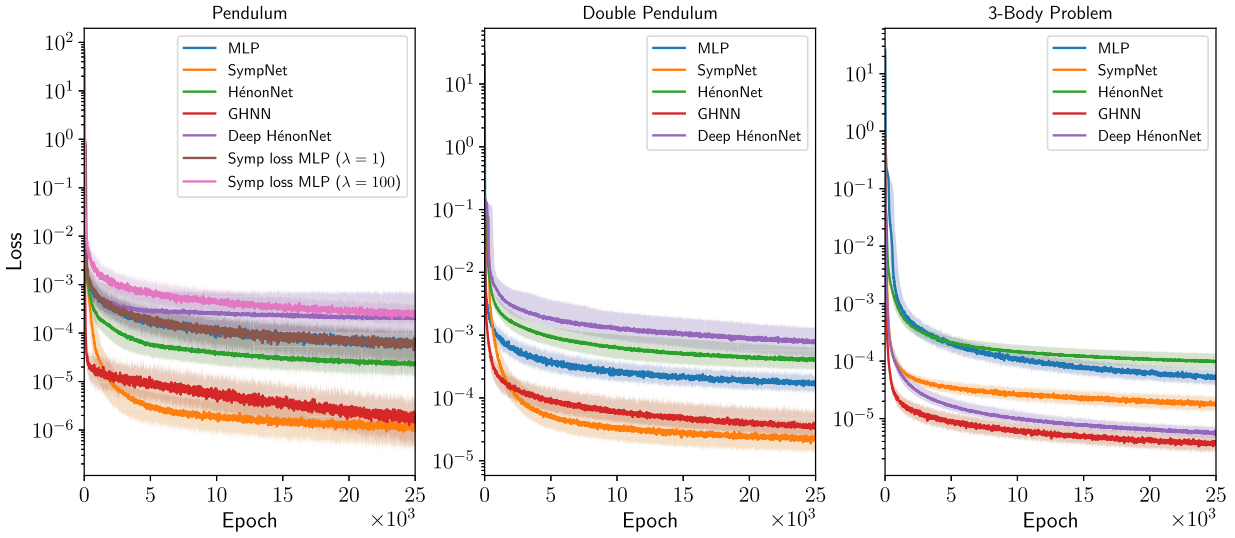

Fig. C.16. Comparison of the validation loss during the entire 25000 epochs of training for all three Hamiltonian systems. The solid lines show the behavior of the median of the 50 neural networks trained per architecture. The lighter colored areas indicate the variations between the differently initialized neural networks.

Data availability

Code and Data can be accessed via GitHub and Zenodo via the links provided at the end of the Implementation section.

References

[1] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations, J. Comput. Phys. 378 (2019) 686–707.
[2] P.G. Breen, C.N. Foley, T. Boekholt, S. Portegies Zwart, Newton versus the machine: solving the chaotic three-body problem using deep neural networks, Mon. Not. R. Astron. Soc. 494 (2) (2020) 2465–2470.
[3] E. Hairer, C. Lubich, G. Wanner, Geometric Numerical Integration, Springer, 2006.
[4] S. Greydanus, M. Dzamba, J. Yosinski, Hamiltonian neural networks, in: Advances in Neural Information Processing Systems, vol. 32, Curran Associates, Inc., 2019.
[5] Z. Chen, J. Zhang, M. Arjovsky, L. Bottou, Symplectic recurrent neural networks, in: 8th International Conference on Learning Representations, ICLR 2020, 2020.
[6] P. Jin, Z. Zhang, A. Zhu, Y. Tang, G. Karniadakis, SympNets: intrinsic structure-preserving symplectic networks for identifying Hamiltonian systems, Neural Netw. 132 (2020) 166–179.
[7] J. Burby, Q. Tang, R. Maulik, Fast neural Poincaré maps for toroidal magnetic fields, Plasma Phys. Control. Fusion 63 (2021) 024001.
[8] S. Xiong, Y. Tong, X. He, S. Yang, C. Yang, B. Zhu, Nonseparable symplectic neural networks, in: 9th International Conference on Learning Representations, ICLR 2021, 2021.
[9] K. Hornik, Approximation capabilities of multilayer feedforward networks, Neural Netw. 4 (2) (1991) 251–257.
[10] D. Turaev, Polynomial approximations of symplectic dynamics and richness of chaos in non-hyperbolic area-preserving maps, Nonlinearity 16 (1) (2002) 123.
[11] P. Horn, V. Saz Ulibarrena, B. Koren, S. Portegies Zwart, Structure-preserving neural networks for the N-body problem, in: ECCOMAS2022, 2022.
[12] W. E, A proposal on machine learning via dynamical systems, Commun. Math. Stat. 5 (2017) 1–11.
[13] E. Haber, L. Ruthotto, Stable architectures for deep neural networks, Inverse Probl. 34 (1) (2017) 014004.
[14] R.T.Q. Chen, Y. Rubanova, J. Bettencourt, D. Duvenaud, Neural ordinary differential equations, in: Advances in Neural Information Processing Systems, vol. 31, Curran Associates, Inc., 2018.
[15] H. Yoshida, Construction of higher order symplectic integrators, Phys. Lett. A 150 (5) (1990) 262–268.
[16] M. Telgarsky, Representation benefits of deep feedforward networks, arXiv:1509.08101, 2015.
[17] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, S. Chintala, PyTorch: an imperative style, high-performance deep learning library, in: Advances in Neural Information Processing Systems, vol. 32, Curran Associates, Inc., 2019, pp. 8024–8035.
[18] D. Kingma, J. Ba, Adam: a method for stochastic optimization, in: 3rd International Conference on Learning Representations, ICLR 2015, 2015.
[19] W. Kutta, Beitrag zur näherungsweisen Integration totaler Differentialgleichungen, Z. Math. Phys. 46 (1901) 435–453.
[20] D.C. Heggie, R.D. Mathieu, Standardised units and time scales, in: The Use of Supercomputers in Stellar Dynamics, vol. 267, Springer, 1986, pp. 233–235.
[21] T. Boekholt, S. Portegies Zwart, On the reliability of N-body simulations, Comput. Astrophys. Cosmol. 2 (2015).